

Squad agêntica: do primeiro comando ao dia a dia

Como montar e operar um time de agentes de IA com o BMAD: instalação, base do projeto e rotina de desenvolvimento.

Anderson Gutierres

Forward Deployed Engineer

BMAD-METHOD

github.com/bmad-code-org/BMAD-METHOD

SLIDES E PROJETO DE EXEMPLO

github.com/andersou/squad-agentica-bmad

Um assistente sozinho escreve código, mas não sustenta um projeto

ONDE UM CHAT GENÉRICO VAI BEM

- Escrever funções isoladas
- Corrigir erros de compilação
- Refatorar trechos de código
- Acelerar protótipos

ONDE ELE SE PERDE

- O contexto some entre sessões
- Decisões de arquitetura mudam a cada prompt
- Sem critério de aceite, sem estratégia de testes
- Ninguém lembra por que algo foi decidido

O problema não é o modelo. É a falta de estrutura em volta dele.

Squad agêntica é um time de papéis, não um chatbot

01

Papéis

Cada agente tem uma responsabilidade e um menu próprio de workflows: analista, PM, arquiteto, UX, dev.

02

Artefatos

Brief, PRD, arquitetura e stories ficam em arquivos versionados. Cada documento vira o contexto da fase seguinte.

03

Gates

Você revisa e aprova antes de avançar. Checagens como o gate do sprint planning e o code review seguram a qualidade.

O BMAD-METHOD é uma implementação open source desse modelo. O humano continua decidindo; os agentes executam com contexto.

Cinco agentes, cada um invocado como skill

AGENTE	SKILL	FAZ O QUÊ
Mary · Analista	<code>bmad-agent-analyst</code>	Brainstorming, pesquisa, product brief
John · PM	<code>bmad-agent-pm</code>	PRD, épicos e stories, correção de rota
Winston · Arquiteto	<code>bmad-agent-architect</code>	Arquitetura e gate de prontidão
Sally · UX	<code>bmad-agent-ux-designer</code>	Design de experiência
Amelia · Dev	<code>bmad-agent-dev</code>	Build: spec, código, testes e revisão

Não existe Scrum Master separado: a Dev implementa cada story com `bmad-build`. A Tech Writer (Paige) saiu na 6.11.

Quatro fases, e cada documento vira contexto da próxima

1 · ANÁLISE · OPCIONAL

Entender o problema

Brainstorming, pesquisa, product brief ou PRFAQ.

2 · PLANEJAMENTO

O que construir

PRD com requisitos e, se houver interface, UX.

3 · SOLUÇÃO

Como construir

Arquitetura, épicos e stories.

4 · IMPLEMENTAÇÃO

Construir

Gate e sprint; depois, por story, build e review.

CAMINHO CURTO

Intenção já clara vai direto para `bmad-spec` e `bmad-build`, sem passar pelas fases 1–3.

01

Primeiros passos

Instalar, perguntar ao `bmad-help` e escolher o caminho certo.

Cinco coisas antes do primeiro comando

ITEM	POR QUÊ
Node.js 20.12+	Exigido pelo instalador, que roda via <code>npx</code>
Git	Artefatos e código viajam juntos no histórico
IDE com agente de IA	Claude Code, Cursor e outras. Para ver todas: <code>npx bmad-method install --list-tools</code>
uv	Obrigatório: <code>bmad-build</code> e outros skills rodam scripts com <code>uv run</code>
Uma ideia pequena	O primeiro projeto é para aprender o fluxo, não para salvar o trimestre

Um comando instala a squad dentro do seu repositório

```
$ npx bmad-method install
```

- ◆ Diretório → ./api-tarefas
- ◆ Módulos → BMad Method (bmm)
- ◆ Tudo estável? → Sim
- ◆ Ferramenta → claude-code
- ◆ Config → nome, idioma, pasta

```
api-tarefas/  
├ _bmad/          agentes, workflows  
└ _bmad-output/  seus artefatos
```

Na v6, os agentes e workflows são **skills** que a IDE reconhece pelo nome `bmad-*`. No Claude Code, chame com barra: `/bmad-help`.

Travou? O bmad-help lê o projeto e diz o próximo passo

```
> /bmad-help tenho uma ideia de API  
de tarefas, por onde começo?
```

```
→ caminho recomendado
```

```
→ primeira tarefa obrigatória
```

```
→ como segue o resto do processo
```

- Inspeciona o que já foi feito no projeto
- Mostra as opções dos módulos instalados
- Recomenda a próxima tarefa obrigatória
- Roda sozinho ao fim dos workflows de planejamento

Escolha o caminho pelo quanto a intenção já está definida

SITUAÇÃO	CAMINHO	DOCUMENTOS
Mudança pequena, cabe numa sessão (~500 linhas)	<code>bmad-build</code> direto	Só a spec que o build escreve
Intenção clara, maior que uma sessão	<code>bmad-spec</code> , depois um build por story	SPEC.md e stories.yaml
Produto com várias pessoas e épicos	PRD, arquitetura e épicos	PRD + arquitetura + épicos + sprint

Na dúvida, pergunte ao `bmad-help` . Em projeto existente, comece com `bmad-project-context` para registrar o que o código já faz.

02

Criar a base

O exemplo do começo ao fim: uma **API REST de tarefas** com tags e prazos, pelo caminho completo: PRD, arquitetura e épicos.

Primeiro o problema, depois os requisitos: brief e PRD

ANALISTA • OPCIONAL

`bmad-product-brief`

Uma API de tarefas para times pequenos, que permita criar, etiquetar e filtrar por prazo.

Saída: brief.md, addendum.md, .memlog.md

PM • OBRIGATÓRIO

`bmad-prd` → Create

Uma descoberta guiada transforma o brief em requisitos testáveis, métricas de sucesso e escopo da v1.

Saída: prd.md, .memlog.md e três revisões automáticas

Revise o escopo no PRD, que é o momento mais barato de mudar. Se houver interface, rode `bmad-ux` em seguida.

A API do exemplo não tem interface, então pulamos esse passo.

Arquitetura: stack, dados, erros e testes decididos uma vez só

ARQUITETO · bmad-architecture

- Stack e versões
- Modelo de dados: tarefa, tag, prazo
- Padrões de API e de erros
- Estratégia de testes

Saída: ARCHITECTURE-SPINE.md e quatro revisões

RECOMENDADO

AGENTS.md, a constituição do projeto

`bmad-project-context` grava um bloco verificado no AGENTS.md, que todo agente lê antes de agir. O CLAUDE.md só aponta para ele.

Na v6, épicos e stories nascem depois da arquitetura

PM · bmad-create-epics-and-stories

Usa o PRD e a arquitetura juntos, então as stories já levam em conta o banco, os padrões de API e a stack.

No exemplo (epics.md real):

Épico 1: Cadastrar e manter tarefas · 4 stories

Épico 2: Consultar prazos por janela e tag · 3 stories

GATE · bmad-sprint-planning

Antes de gerar o sprint, confere se PRD, arquitetura e stories contam a mesma história.

PASS

CONCERNS

FAIL

Implemente com PASS, ou com CONCERNS que você leu e aceitou.

Ao fim da base, o plano inteiro está versionado no Git

```
api-tarefas/  
├ .claude/skills/ ← 29 skills  
├ _bmad/  
├ AGENTS.md ← contexto verificado  
├ CLAUDE.md ← @AGENTS.md  
└ _bmad-output/  
  ├── planning-artifacts/  
  │ ├── briefs/.../brief.md  
  │ ├── prds/.../prd.md  
  │ ├── architecture/.../ARCHITECTURE-SPINE.md  
  │ ├── epics.md  
  │ └ sprint-planning-gate-....md  
  └ implementation-artifacts/ ← fase 4  
    ├── sprint-status.yaml  
    └ spec-1-1-criar-e-listar-tarefas.md
```

Os artefatos são texto: dá para revisar em PR, e qualquer agente consegue ler em um chat novo.

O contexto sai da sua cabeça e vai para o repositório.

Estrutura real da v6.12.1 com Claude Code (tag bmad-v6.12.1). Os "..." são pastas datadas por execução.

No código existente, mostre à squad como o projeto já funciona

01

Limpe o plano antigo

Arquive PRDs, épicos e stories já concluídos para que nenhum agente trate um plano velho como atual.

02

Gere o contexto do projeto

`bmad-project-context`

Lê o código, pergunta o que nenhum scan acha e grava um bloco verificado no AGENTS.md.

03

Mantenha o contexto vivo

`refresh · record · audit`

Depois de mudança real, refresh. Quando o agente repete um erro, record.

Na dúvida, pergunte: `bmad-help tenho uma API existente, por onde começo?` O `bmad-project-context` nunca commita: tudo fica no working tree para você revisar.

No brownfield, o tamanho da mudança define o caminho

MUDANÇA PEQUENA

`bmad-build`

- Investiga o código e o contexto antes de planejar
- Escreve a spec, implementa e revisa
- Só pergunta o que a evidência não resolve

MUDANÇA GRANDE

BMad Method, com o rigor necessário

- O PRD parte da documentação existente
- A arquitetura escaneia o código antes de decidir
- UX só se houver tela nova ou padrão novo

Uma sessão de build é um objetivo: cerca de 500 linhas em poucos arquivos. Passou disso, planeje antes.

03

Operar no dia a dia

Uma story por vez. Um chat novo para cada workflow.

O ciclo: gate uma vez, depois build, review e repetir

UMA VEZ `bmad-sprint-planning` roda o gate e gera o sprint-status.yaml com épicos e stories

1

`bmad-build`

Investiga, escreve a spec da story, implementa, revisa e corrige. Deixa a story em review.



2

`bmad-code-review`

Revisores em paralelo; a triagem manda cada achado para patch, defer ou decisão.



3

Story em done

O review fecha a story no sprint-status, e o commit leva código e spec juntos.

↻ Story em done, próxima story

Fim do épico: `bmad-retrospective`

Fora do ciclo: mudança grande, ajuste pequeno e modo autônomo

MUDANÇA GRANDE

`bmad-correct-course`

Mudança significativa no meio do sprint.
O resultado é um plano atualizado ou redirecionado.

AJUSTE PEQUENO

`bmad-build`

Bug ou ajuste fora do backlog. A mesma skill da story, sem PRD nem épico.

SEM SUPERVISÃO

`bmad-build-auto`

O build sem você no meio, pensado para orquestradores. Para quando a intenção fica ambígua.

Viu `quick-dev`, `create-story` ou `dev-story` em material antigo? Desde a 6.11, é tudo `bmad-build`.

Cinco erros que viram vibe coding com passos extras

ERRO

CORREÇÃO

Pedir código à Dev sem PRD e arquitetura

Escolha o caminho. Para o que é pequeno, `bmad-build` direto

Reusar o mesmo chat para vários workflows

Um chat novo por workflow, porque o contexto está nos arquivos

Aprovar artefatos sem ler

Os gates só funcionam se alguém ler o que eles produzem

Stories grandes, misturando épicos

Uma story é uma entrega pequena e testável

Ignorar o gate do sprint

Leia o veredito do `bmad-sprint-planning` antes da primeira story

04

Na prática: do PRD à retrospectiva

A API de tarefas refeita do zero na v6.12.1. As perguntas do PM são **ilustrativas**; os terminais resumem as saídas reais.

O PRD nasce de uma conversa: o PM pergunta antes de escrever

> bmad-prd API de tarefas para times
pequenos, com tags e prazos.

PM ► Quem usa e o que dói hoje?

> Devs que perdem prazos em planilha.

PM ► O filtro por prazo usa o fuso
do usuário ou do servidor?

prd.md real · resumido

3. Glossário

Hoje: data corrente em America/Sao_Paulo

Janela: vencidas · hoje · próximos 7 dias

4. Funcionalidades

FR-1 Criar tarefa FR-4 Listagem geral

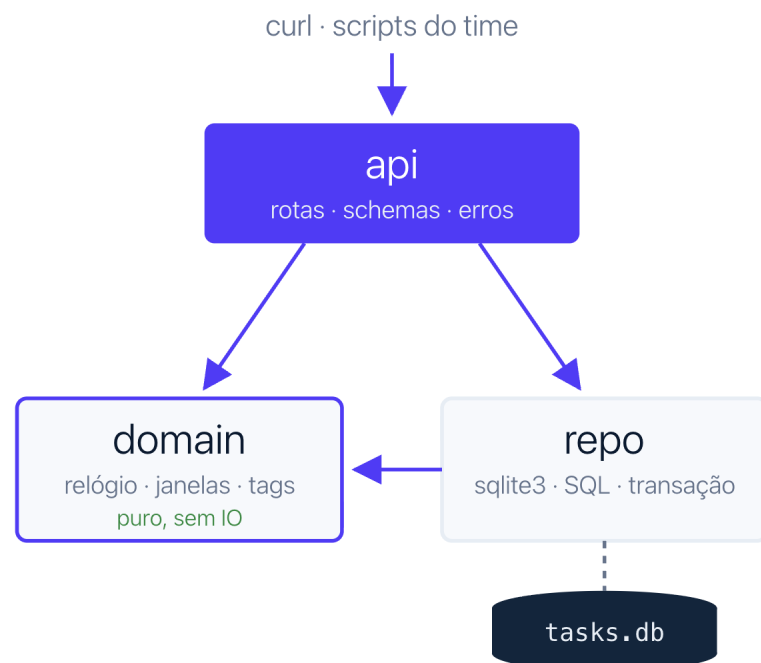
FR-2 Editar tarefa FR-5 Tags e filtro

FR-3 Excluir tarefa FR-6 Janelas de prazo

6. Escopo · fora da v1

- Autenticação, usuários e permissões
- Paginação (fica no backlog)

A arquitetura fixa o que dois agentes decidiriam diferente



Setas = quem pode depender de quem. Nenhuma outra dependência é permitida.

Python 3.14

FastAPI 0.142

sqlite3

pytest

uv

AD-2

Relógio injetado

`api.agora()` é a única leitura do relógio, e `domain.hoje()` converte para São Paulo.

`date.today()` é proibido.

AD-7

Contrato HTTP

POST 201, GET 200, PATCH 200, DELETE 204. Erros no formato padrão `{"detail": ...}`.

O gate deu CONCERNS: a story 1.1 contradizia a convenção de testes

- bmad-sprint-planning
- FR-1 a FR-6 e NFR-1 a NFR-5 cobertos por stories
- Nenhuma story depende de outra futura
- Achado: o AC de persistência da 1.1 pede outro TestClient, e o AGENTS.md proíbe
- Veredito: CONCERNS · 3 opções
 - 1: corrigir o AC
- AC corrigido · sprint-status gerado

O QUE O GATE ACHOU

CONCERNS

Quem implementasse a 1.1 teria que decidir sozinho qual regra vale. O gate transformou isso em pergunta.

Você escolhe: corrigir o AC, mudar a convenção ou seguir. Aqui, o AC foi corrigido antes do sprint.

Story 1.1: o bmad-build escreve a spec antes do código

```
spec-1-1-criar-e-listar-tarefas.md · resumida
status: done · review_loop_iteration: 2
## Intent
0 time não consegue recadastrar as
tarefas da planilha nem vê-las por prazo
## Boundaries · Never
PATCH, DELETE, janela, ORM, date.today()
## I/O & Edge-Case Matrix · 5 cenários
## Review Triage Log
## Conferência AD-1 a AD-9
```

```
> bmad-build story 1.1
▸ spec: intenção, limites, matriz
▸ implementa api · domain · repo
▸ triagem: 19 achados, 9 rejeitados
  8 patch · 1 pergunta · 1 adiado
▸ confere AD-1 a AD-9 no diff
▸ Status: review
$ git commit -m "feat: criar e
  listar tarefas (story 1.1)"
```

O code review derrubou o único adiamento da story 1.1

```
> (chat novo) bmad-code-review
> 4 camadas: blind-hunter, edge-case,
  verification-gap, acceptance
> Adiado no build: "não dá para testar
  o desempate id ASC" → premissa falsa
> Teste novo, provado por mutação
> Surrogate UTF-16 → pergunta → rejeitado
> Status: done
$ git commit -m "fix: achados da
  code review da story 1.1 (...)"
```

```
sprint-status.yaml real · fim da sprint
development_status:
  epic-1: in-progress
    1-1-criar-e-listar-tarefas: done
    1-2-etiquetar-tarefas-na-...: done
    1-3-excluir-tarefa: done
    1-4-editar-e-concluir-tarefa: done
  epic-1-retrospective: done
  epic-2: in-progress
    2-1-filtrar-a-listagem-...: done
    2-2-consultar-pendentes-...: done
    2-3-documentar-as-chamadas-...: done
  epic-2-retrospective: done
```

A retrospectiva achou o que nenhuma story via sozinha

```
› bmad-retrospective épico 2
› 3/3 stories done · veredito:
  accepted-with-open-items
› R1: o relógio fixo dos testes cai
  no dia real; vazar o relógio passa
  com os 131 testes verdes
› R2: a guarda não pega CURRENT_DATE
  no SQL
› Prova por mutação, não por opinião
$ git commit -m "fix: action items 1 a 4
  da retro do épico 2" # 137 passed
```

DA V6.10 PARA A V6.12.1

As lições da retro anterior viraram regra no AGENTS.md:
corrigir achado baixo, decisão vira pergunta, revisar cada AD.

Mesmo assim, a primeira passada de review ainda rejeitou
achados baixos.

Regra escrita não basta: o workflow precisa carregá-la.

Checklist: cinco passos do primeiro dia, três lições das retrospectivas

- | | |
|--|---|
| ☐ SETUP
Node.js 20.12+, uv, Git e a IDE com IA aberta no repositório | ☐ INSTALAR
<code>npx bmad-method install</code> com o módulo BMad Method |
| ☐ ORIENTAR
<code>bmad-help</code> para saber qual caminho seguir e por onde começar | ☐ PLANEJAR
PRD e arquitetura aprovados e o gate do sprint lido |
| ☐ EXECUTAR
A primeira story pelo <code>bmad-build</code> e pelo <code>bmad-code-review</code> | ☐ REGISTRAR
Lição de retrospectiva vira regra no AGENTS.md |
| ☐ REVISAR
Achado com correção clara é corrigido na story; decisão vira pergunta | ☐ TESTAR
Teste que passa por coincidência não prova nada |

OBRIGADO · PERGUNTAS?

Planejar com agentes é barato. Corrigir sem plano, não.

Anderson Gutierres

Forward Deployed Engineer

anderson.gutierres@br.experian.com

SLIDES E PROJETO DE EXEMPLO

github.com/andersou/squad-agentica-bmad

DOCS

docs.bmad-method.org

CÓDIGO

github.com/bmad-code-org/BMAD-METHOD

COMUNIDADE

discord.gg/gk8jAdXWmj